
Basic Linux Commands and Tools

Getting familiar with Linux

Contents

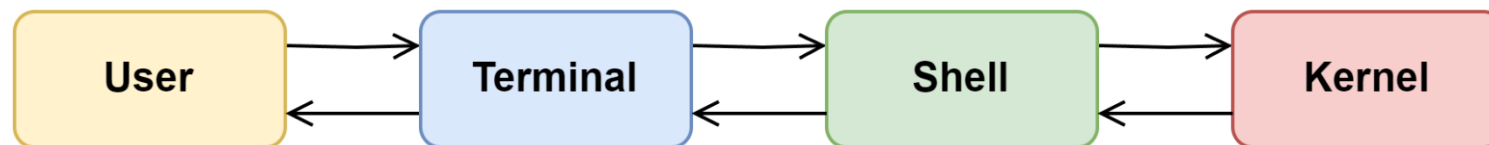
- Linux Commands
- Git
- Visual Studio Code
- Hand-on Labs
 - Hello, xv6
- Appendix
 - Vim
 - GCC
 - Makefile

Basic Linux Command and Tools

#Linux Commands

Bridge between Users and OS

- Shell
 - User interfaces for access to an operating system's services.
 - Shell receives User's command and passes on to the Kernel and get responses.
 - sh (Bourne Shell), bash (Bourne Again Shell)
 - cmd, powershell (Window)
 - zsh (z shell, macOS)
- Terminal
 - Device that accesses the system and controls input/output.
 - Windows Terminal, iterm2, etc.



Basic Linux Commands

- Shell provides some basic system programs.
- Typing those commands from terminal, shell interprets them and provides useful services.

ls	show list
cd	change directory
mkdir	make directory
rmdir	remove directory
cp	copy file
rm	remove file
mv	rename(move) file

ls

- Prints out files and directories on current working directory
- Options
 - -a: show all list including hidden items.
 - -l: provides detail information.

```
splab@DESKTOP-HPBRE5B:~/os$ ls
hello hello.c hello_world main.cpp print.cpp project
splab@DESKTOP-HPBRE5B:~/os$ ls -al
total 60
drwxr-xr-x  3 splab splab 4096 Mar 10 17:44 . (current directory)
drwxr-x--- 16 splab splab 4096 Mar 10 18:01 .. (parent directory)
-rwxr-xr-x  1 splab splab 15960 Mar 10 17:26 hello
-rw-r--r--  1 splab splab   65 Mar 10 17:26 hello.c
-rwxr-xr-x  1 splab splab 16424 Mar 10 17:32 hello_world
-rw-r--r--  1 splab splab   63 Mar 10 17:31 main.cpp
-rw-r--r--  1 splab splab   75 Mar 10 17:32 print.cpp
drwxr-xr-x  6 splab splab 4096 Mar 10 17:55 project
splab@DESKTOP-HPBRE5B:~/os$ █
```

mkdir, rmdir, cd

- mkdir [*directory name*]
 - Make a directory.
- rmdir [*directory name*]
 - Remove the directory. (must be empty)
- cd [*directory name or pathname*]
 - Change current working directory.
 - e.g. \$ `cd ..` : go to the parent directory

mkdir, rmdir, cd

```
splab@DESKTOP-HPBRE5B:~/os$ ls
hello  hello.c  hello_world  main.cpp  print.cpp
splab@DESKTOP-HPBRE5B:~/os$ mkdir project
splab@DESKTOP-HPBRE5B:~/os$ ls
hello  hello.c  hello_world  main.cpp  print.cpp  project
splab@DESKTOP-HPBRE5B:~/os$ cd project/
splab@DESKTOP-HPBRE5B:~/os/project$ ls
splab@DESKTOP-HPBRE5B:~/os/project$ cd ..
splab@DESKTOP-HPBRE5B:~/os$ rmdir project/
splab@DESKTOP-HPBRE5B:~/os$ ls
hello  hello.c  hello_world  main.cpp  print.cpp
splab@DESKTOP-HPBRE5B:~/os$
```


cp, rm, mv

- cp [A file] [B file]
 - Copy file A file to B file.
- rm [target file]
 - Remove file.
- mv [A file] [B file]
 - Rename file name A to B.
- mv [target file] [directory name or path]
 - Move file.

cp, rm, mv

```
splab@DESKTOP-HPBRE5B:~/os$ mkdir project
splab@DESKTOP-HPBRE5B:~/os$ ls
project  test.txt
splab@DESKTOP-HPBRE5B:~/os$ cat test.txt
hello
splab@DESKTOP-HPBRE5B:~/os$ mv test.txt project/
splab@DESKTOP-HPBRE5B:~/os$ ls
project
splab@DESKTOP-HPBRE5B:~/os$ cd project/
splab@DESKTOP-HPBRE5B:~/os/project$ ls
test.txt
splab@DESKTOP-HPBRE5B:~/os/project$ mv test.txt test2.txt
splab@DESKTOP-HPBRE5B:~/os/project$ ls
test2.txt
splab@DESKTOP-HPBRE5B:~/os/project$ cat test2.txt
hello
splab@DESKTOP-HPBRE5B:~/os/project$ mv test2.txt ..
splab@DESKTOP-HPBRE5B:~/os/project$ ls
splab@DESKTOP-HPBRE5B:~/os/project$ cd ..
splab@DESKTOP-HPBRE5B:~/os$ ls
project  test2.txt
splab@DESKTOP-HPBRE5B:~/os$
```

Other Commands

- There are lots of other useful commands provided from Shell.
 - <https://ss64.com/bash>
 - https://www.mireene.com/webimg/linux_tip1.htm

pwd	Print out current path location.
whereis	Find out files
who	Print out the accessed users
touch	Create an empty file
cat	Print out the contents of file
tar	Compress the file
passwd	change the user's password
chmod	change the privilege of file

Basic Linux Command and Tools

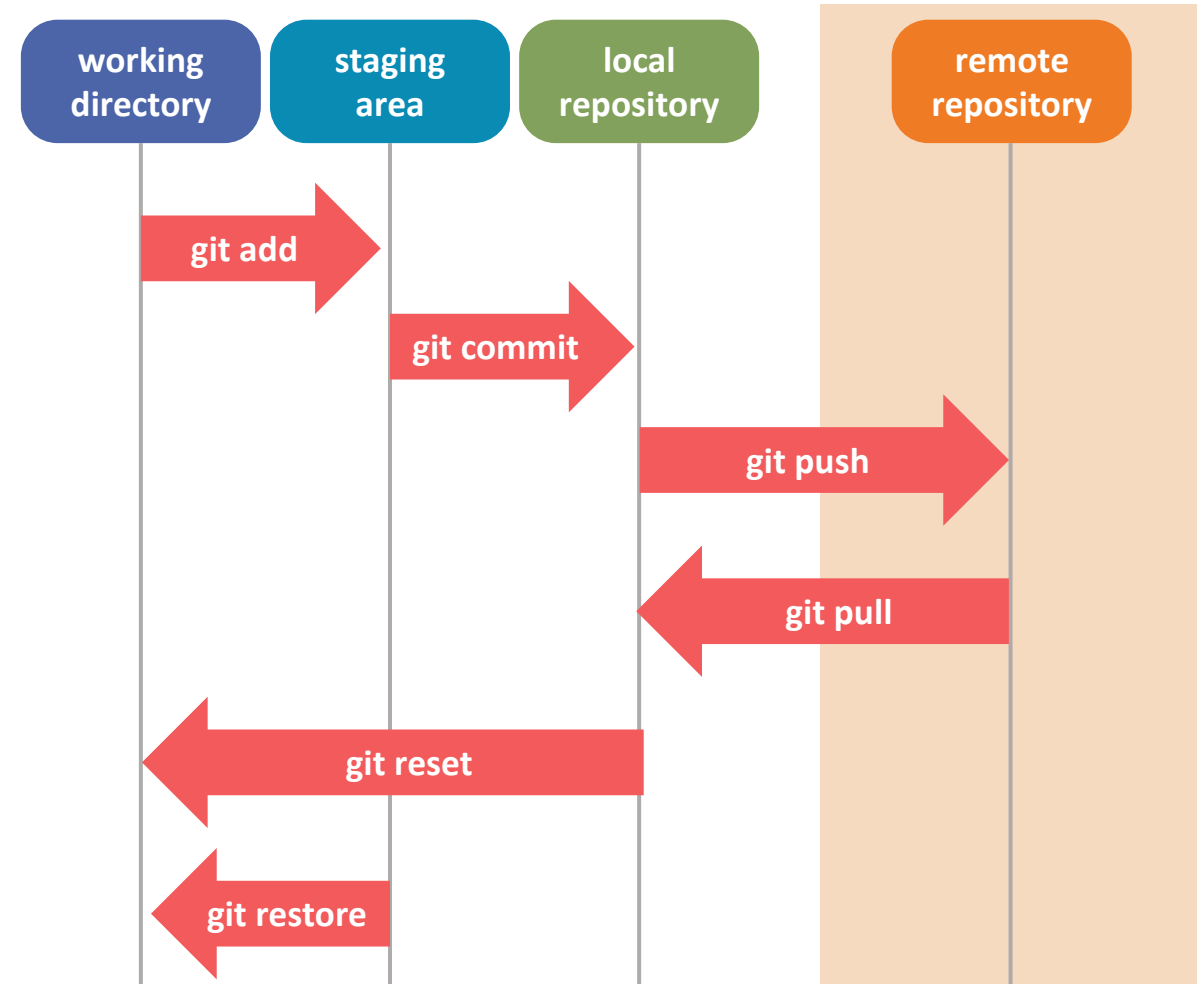
#Git

What is Git?

- **Git**
 - Distributed Version Control System
 - Tracks and manages changes to source code during software development
 - Developed by Linus Torvalds (Linux kernel developer) in 2005
- **Key Features**
 - Distributed architecture allows offline work with a full local repository copy
 - Optimized for speed and highly efficient storage structure
 - Support **safe, independent workspaces** through branching

Git Workflow

- **Stage:** select changes to commit
- **Commit:** apply changes to local repository
- **Push:** upload local changes to remote repository
- **Pull:** download and apply changes from remote changes
- **Reset:** return to a previous state, “commit” of local repository
- **Restore:** discard uncommitted changes in working directory



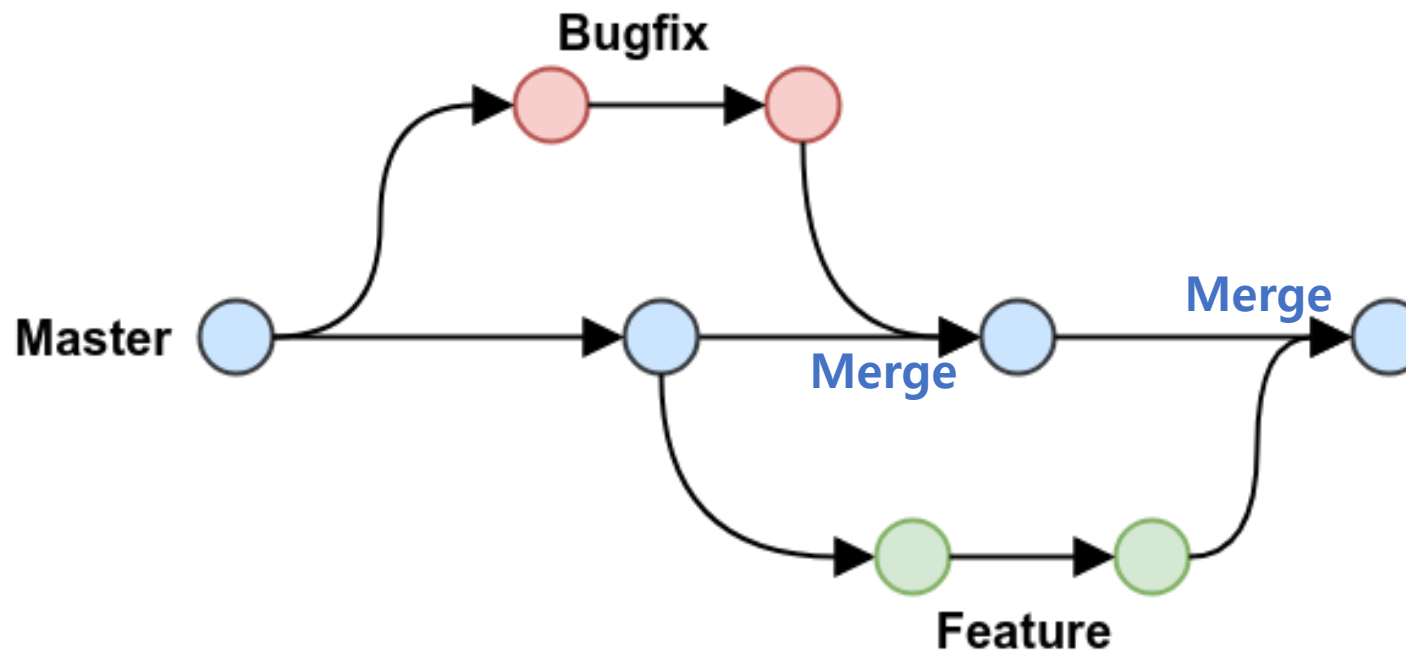
Git Branch

- **Branch**

- An independent line of development parallel to the main workspace
- Enables safe experimentation, new features, or bug fixes without affecting stable code

- **Merge**

- The process of integrating changes from one branch back into another



Git Installation and Setup

- **Installation**

```
$ sudo apt install git
```

- **Initial Configuration (User Information)**

```
$ git config --global user.name "Your Name"  
$ git config --global user.email "your.email@example.com"
```

- **Verify Configuration**

```
$ git config --list
```


Basic Git Commands (1)

- **Repository Creation**

```
$ git init                # Create a new repository in current directory
$ git clone [URL]         # Clone a remote repository
```

- **Staging and Committing Changes**

```
$ git status              # Check working directory status
$ git add [filename]      # Stage specific file
$ git add .               # Stage all changes
$ git commit -m "message" # Commit changes
```

- **Undoing Changes**

```
$ git restore [filename]  # Discard changes
$ git reset [commit_id]  # return to specific commit
```

Basic Git Commands (2)

- Remote Repository Management

```
$ git remote add origin [URL]          # Add remote repository
$ git push -u origin [branch]          # Push to remote repository
$ git pull origin [branch]             # Pull from remote repository
```

- Branch Management

```
$ git branch                          # List branches
$ git branch [branch-name]            # Create new branch
$ git switch [branch-name]            # Switch to branch
$ git switch -c [branch-name]          # Create and switch to new branch
$ git merge [branch-name]              # Merge branch into current branch
```

Additional Resources

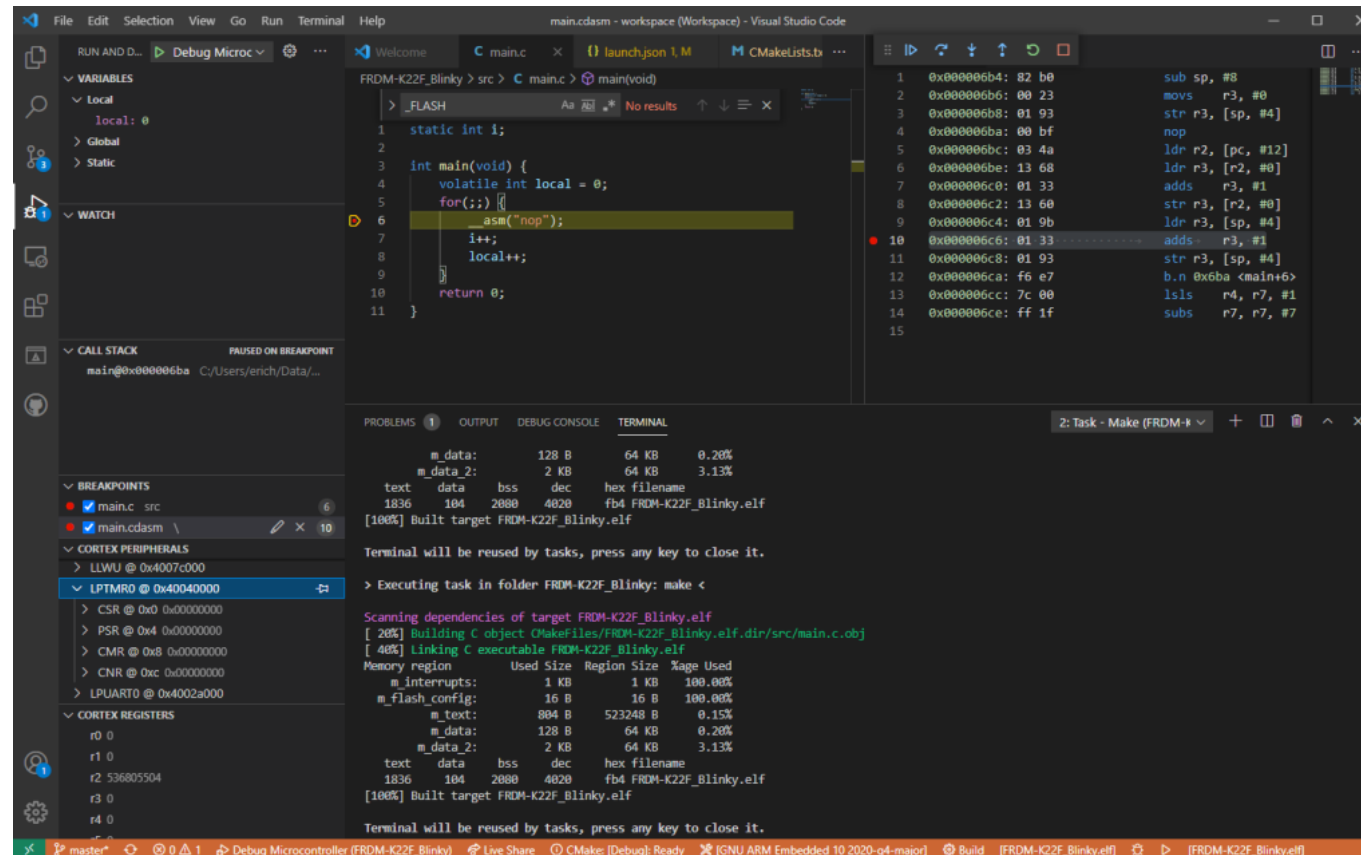
- There are many more Git commands to explore, so try them yourself!
- Once you get comfortable with Git, you'll find it incredibly valuable for your projects.
- Official Documentation and Tutorials
 - [Git Official Documentation](#)
 - [Git Simple Guide](#)
 - [Interactive Git Tutorial](#)

Basic Linux Command and Tools

#Visual Studio Code

Visual Studio Code

- Lightweight but powerful source code editor
- Extensive marketplace for extensions
- Cross-platform (Windows, macOS, Linux)
- Highly customizable for different programming languages



Visual Studio Code Installation

- Windows
 - Download the installer from <https://code.visualstudio.com/>
 - Run the installer and follow the instructions
 - **Make sure to check "Add to PATH" option**
- MacOS
 - Download .dmg file from <https://code.visualstudio.com/>
 - Drag VS Code.app to the Applications folder
 - Optional: Add to dock for easy access

Using 'code' Command

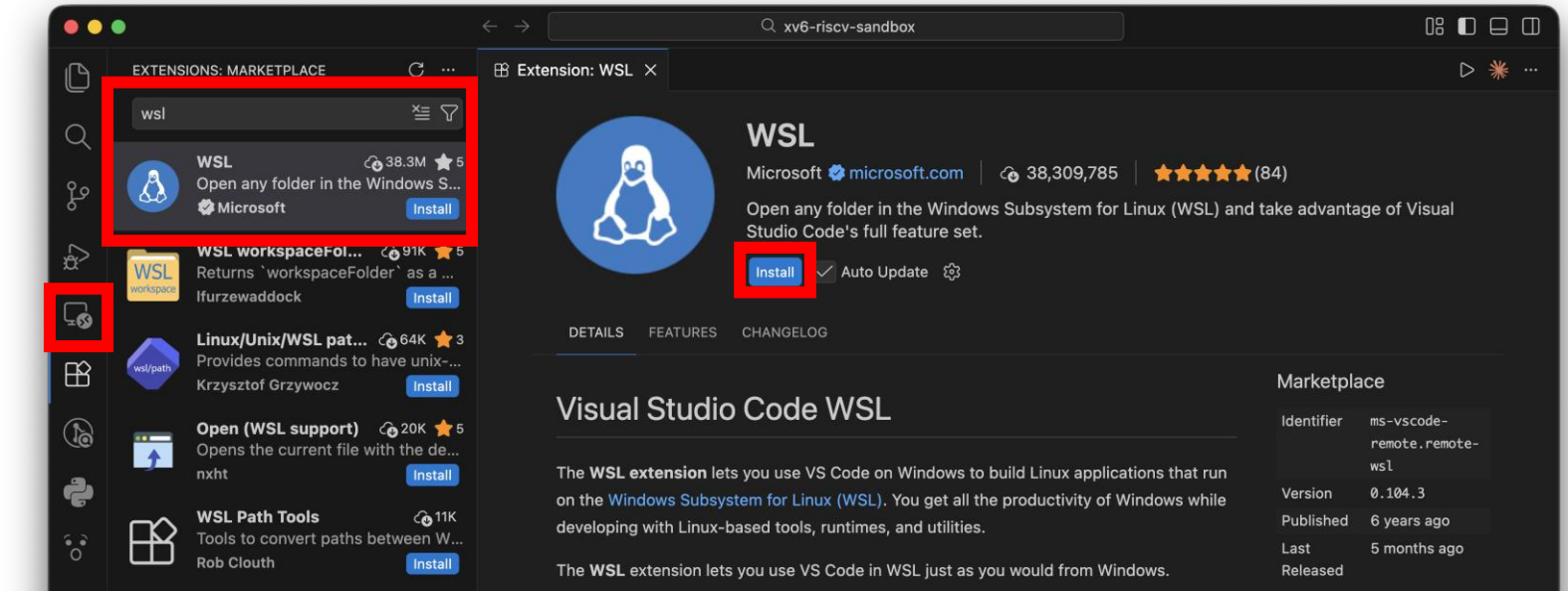
- After installation, you can use the `code` command to open VS Code in terminal

```
$ code                # Open VS Code
$ code .              # Open current directory in VS Code
$ code [file or directory] # Open specific file or directory
$ code -n             # Open a new window
$ code -r [file]      # Open file but reuse window
```

- If `code` command is not working:
 - Open VS Code → Command Palette (Ctrl+Shift+P or Cmd+Shift+P)
 - Type "shell command" → Select "Install 'code' command in PATH"
 - For WSL2 users, recommend to download "WSL" extension (Next slide)

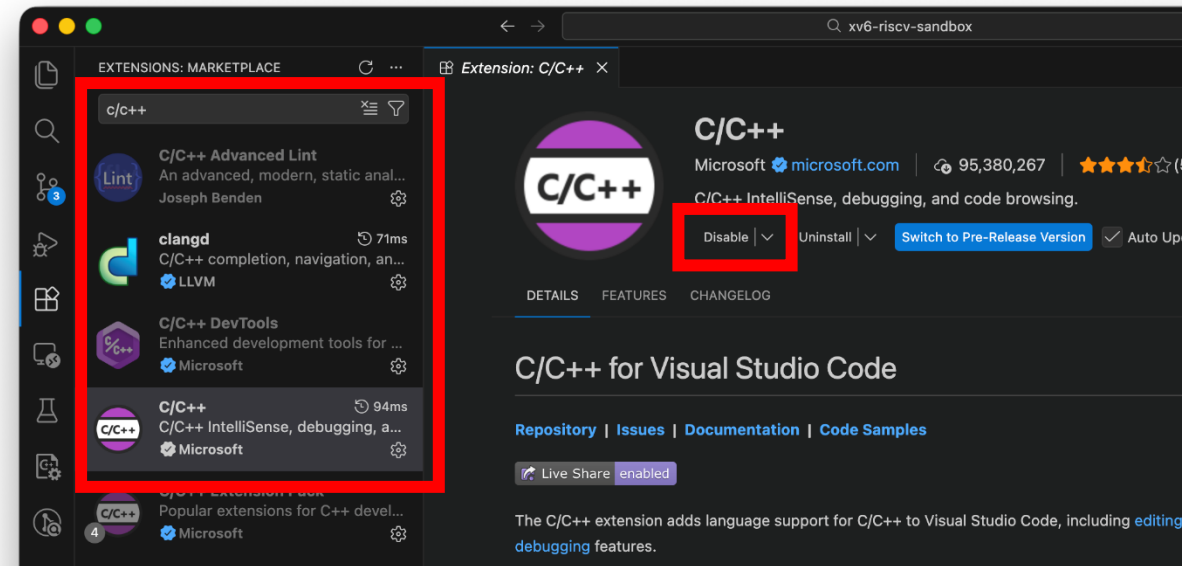
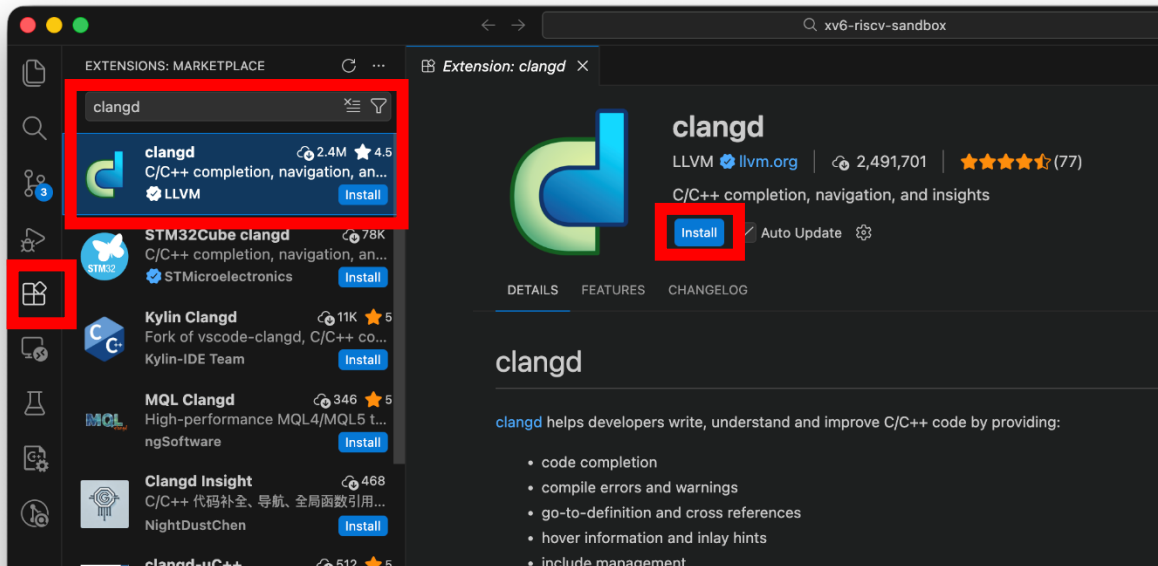
Install WSL extension

- Runs VS Code on Windows with a Linux backend
- Terminal, debugger, and extensions all execute inside Linux
- **Windows users only**, macOS/Linux users can skip



Install clangd extension

- clangd
 - A powerful C/C++ language server based on LLVM
 - Smarter and much lighter than the default Microsoft C/C++ IntelliSense
 - Completely replacing the default C/C++ engine
- Install clangd extension and disable c/c++ extension (if exists)



Generate compile_commands.json

- **compile_commands.json**
 - A file containing exact compiler flags and dependencies for each source file.
 - clangd relies heavily on this file to understand the project structure accurately.
- **Generate compile_commands.json using bear**

- Linux

```
$ sudo apt update
$ sudo apt install bear
$ make clean
$ bear -- make qemu
```

- MacOS

```
$ brew update
$ brew install bear
$ make clean
$ bear -- make qemu
```

Keyboard Shortcuts

- General

- **Ctrl+Shift+P / Cmd+Shift+P**

Command Palette

- **Ctrl+P / Cmd+P**

Quick Open, Go to File

- Editing

- Alt+ ↑ / ↓ / Option+ ↑ / ↓

Move line up/down

- Ctrl+/ / Cmd+/

Toggle line comment

- Navigating

- Ctrl+G / Cmd+G

Go to line

- **Ctrl+Shift+F / Cmd+Shift+F**

Search across all files

- **Ctrl+Shift+O / Cmd+Shift+O**

Navigate to symbols in file

- **Ctrl+T / Cmd+T**

Navigate to symbols across all files

Practice

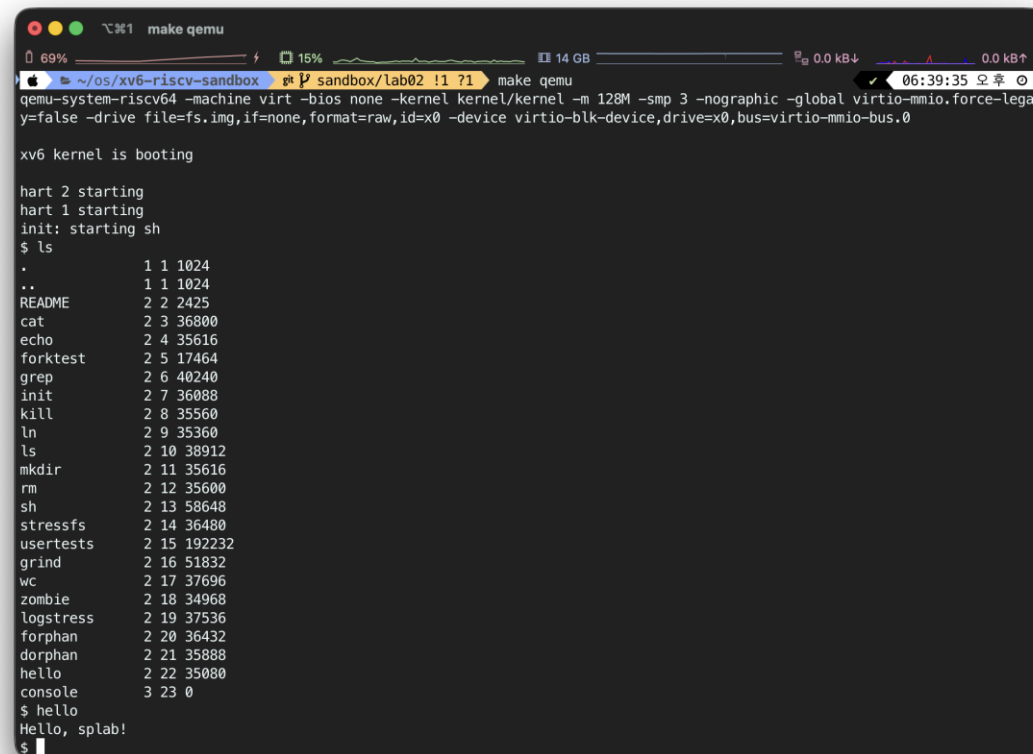
- Open your assignment directory with vscode
 - Hint: code command
- Find "main" function definition in kernel/main.c
 - Hint: Ctrl + P, Ctrl + Shift + O
- Find "proc" structure definition
 - Hint: Ctrl + T
- Find all functions that calling "swtch" function
 - Hint: Ctrl + Shift + F

Hand-on Labs

Hello, xv6

Hello, xv6

- **Goal:** Modify xv6 so that typing hello prints “hello <name>”
- **Branching:** Create a [sandbox/lab02](#) branch in the [xv6-sandbox](#) repository and work within this workspace.
- **Submission:** Push your completed work to the remote repository upon completion.



```
make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2425
cat        2 3 36800
echo       2 4 35616
forktest  2 5 17464
grep       2 6 40240
init       2 7 36088
kill       2 8 35560
ln         2 9 35360
ls         2 10 38912
mkdir      2 11 35616
rm         2 12 35600
sh         2 13 58648
stressfs   2 14 36480
usertests  2 15 192232
grind      2 16 51832
wc         2 17 37696
zombie     2 18 34968
logstress  2 19 37536
forphan    2 20 36432
dorphan    2 21 35888
hello      2 22 35080
console    3 23 0
$ hello
Hello, splab!
$
```

Hello, xv6

- Files to change

```
$ git status --short
```

```
M Makefile           // modified
?? user/hello.c       // new file
```

- Commit and push to your own remote repository

```
$ git add user/Makefile user/hello.c
$ git commit -m "lab02"
$ git push origin sandbox/lab02
```

- TA's code will be uploaded this weekend in [sandbox/lab02](#)
 - Try it first, then check!

Appendix

Basic Linux Command and Tools

Vim

Vim Editor

- vi
 - Visual Text Editor for UNIX
 - Providing 3 separate modes
 - Light-weight but powerful functionality
- vim
 - **vi** improved, extension of vi
 - Providing undo, split windows, syntax coloring, #Vim scripts, etc...
 - Vim is already installed in distributed versions of Linux.

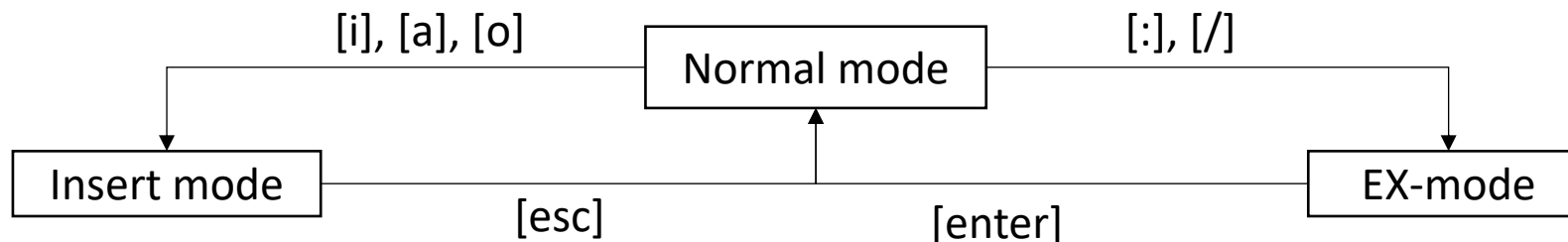
Manual Vim Install

- If vim is not installed, run the command below

```
$ sudo apt update  
$ sudo apt upgrade # If the network is slow, do it next time  
$ sudo apt install vim
```

3 Modes of Vim

- Normal mode.
 - Basic mode when running Vim at first.
 - Code navigation or some manipulation is capable.
- Insert mode
 - You can edit or type the text.
 - Press [i], [o], [a] when Normal mode.
- EX-mode
 - You can save/exit file and search/substitute specific text.
 - Press [:], [/] when Normal mode.



Practice 1

- Write "test.txt" using Vim Editor.
 1. Create test.txt file from terminal, with command `$ vim test.txt`
 2. Press `[i]` to switch to the insert mode.
 3. Write out below <Example> text.
 4. Save and exit, `[:wq]`

<Example>

This is the first vim example file.

The default mode is command mode.

We can write files by typing `[i]` from command mode.

`[i]` means insert.

We can return mode by typing `[esc]`.

We can save and close vim by typing `[:wq]`

`[w]` means write and `[q]` means quit.

Practice 1

```
sp1ab@DESKTOP-HPBRE5B:~/os$ vim test.txt
sp1ab@DESKTOP-HPBRE5B:~/os$ cat test.txt
<Example>
```

```
This is the first vim example file.
The default mode is command mode.
We can write files by typing [i] from command mode.
[i] means insert.
We can return mode by typing [esc].
We can save and close vim by typing [ :wq]
[w] means write and [q] means quit.
```

```
sp1ab@DESKTOP-HPBRE5B:~/os$ █
```

Normal Mode Commands

- Supported commands in normal mode

h, j, k, l	move cursor ←, ↓, ↑, →
0	Move cursor to the first of a line
^	Move cursor to the start of a line (except blanks)
\$	Move cursor to the end of a line
gg	Move cursor to the start of a file
Shift + g	Move cursor to the end of a file

i, a, o	Switch to insert mode
yy	Copy 1 line
5yy	Copy 5 lines
dd	Copy 1 line and delete
5dd	Copy 5 lines and delete
p	Paste copied lines
u	Undo 1 step
Ctrl+r	Redo 1 step (cancel undo)

EX-Mode Commands

- Supported commands in EX-mode

<code>:w</code>	Save file
<code>:q</code>	Exit file
<code>:q!</code>	Exit without saving
<code>:wq</code>	Exit after saving
<code>:5</code>	Move cursor at the start of line 5
<code>:%s/[old]/[new]/g</code>	Substitute [old] words to [new] in a whole file
<code>:%s/[old]/[new]/gc</code>	Substitute [old] words to [new] step by step

<code>:sp(lit) [file]</code>	split the file horizontally
<code>:vs [file]</code>	split the file vertically
Ctrl + w + [←, ↓, ↑, →]	Move to the split screen
(Visual mode) + <code>:norm i//</code>	Insert "//" to the selected lines in visual mode (comments out)
(Visual mode) + <code>:norm 2x</code>	Delete first 2 characters from selected lines in visual mode (Delete comments)

Practice 2

- Modify "test.txt" file.
 - Open test.txt file with command. `$ vim test.txt`
 - Try `[:4]` `[dd]` `[Shift+g]` `[k]` `[p]` `[p]` in normal mode.
 - Press `[u]` to cancel the recent work.
 - Save and exit forcibly, `[:wq!]`.

<Results>

This is the first vim example file.

The default mode is command mode.

We can write files by typing `[i]` from command mode.

We can return mode by typing `[esc]`.

We can save and close vim by typing `[:wq]`

`[i]` means insert.

`[w]` means write and `[q]` means quit.

Practice 3

- Write out basic source using Vim Editor.
 - \$ `vim main.c`
 - Switch to insert mode, [i]
 - Write below <Example> and save/exit, [:wq]

```
<Example>

#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0;
}
```

Vimrc Configuration

- We can configure Vim editor using .vimrc file.
 - `$ vim ~/.vimrc`

```
1 set number           "Show line number"
2 set autoindent        "Auto indent"
3 set smartindent       "Smart indent"
4 set cindent           "C style indent"
5 set shiftwidth=4      "Set auto indent width 4 spaces"
6 set tabstop=4         "Set tab width 4 spaces"
7 set ignorecase        "No classify capital/small characters"
8 set hlsearch          "Hilght serached characters"
9 set nocompatible      "Activate arrow keys"
10 set background=dark  "Highlight dark/light"
11 set fileencodings=utf-8,euc-kr
12 set backspace=indent,eol,start "Activate backspace key"
13 set history=1000     "Record upto 1000 commands"
14 set ruler            "Show cursor position"
15 set nobackup         "No backup file"
16 set title            "Show title"
17 set showmatch        "Show matched brackets"
18 set nowrap           "No wrap"
19 set wmnu
20 █
21 syntax on
```

Vimrc Configuration

```
#include <stdio.h>
int main() {
    printf("Hello\n");
    return 0;
}
```



```
1 #include <stdio.h>
2 int main() {
3     printf("Hello\n");
4     return 0;
5 }
```

Vim Plugin

- Vim can be extended by using plugins.
- Example
 - NERDTree : make you view the file tree.
 - Vim-airline : show you various states like the file name, time and the position of the cursor.
 - Tlist : show you symbols.

```
526 // Press <F1> to display help
527
528 Operators.cpp (/home/bigs/s
529 function
530 finishAsyncRun [Operator
531 require [Scan]
532 getResultSize [Scan]
533 getResultTupleSize [Scan]
534 run [Scan]
535 asyncRun [Scan]
536 getResult [Scan]
537 require [FilterScan]
538 copy2Result [FilterScan]
539 applyFilter [FilterScan]
540 run [FilterScan]
541 asyncRun [FilterScan]
542 createAsyncTasks [Filter
543 filterTask [FilterScan]
544 getResult [Operator]
545 getResultSize [Operator]
546 getResultTupleSize [Operator]
547 require [Join]
548 copy2Result [Join]
549 run [Join]
550 asyncRun [Join]
551 createAsyncTasks [Join]
552 histogramTask [Join]
553 scatteringTask [Join]
554 subJoinTask [Join]
555 copy2Result [SelfJoin]
556 require [SelfJoin]
557 run [SelfJoin]
558 createAsyncTasks [SelfJoin]
559 run [Checksum]
560 asyncRun [Checksum]
561 createAsyncTasks [Checksum]
562 finishAsyncRun [Checksum]

526 __sync_synchronize();
527 // int length = limit/(taskNum);
528 for (int i=0; i<cntTaskLeft; i++) {
529     int startLeft = i*lengthLeft;
530     if (i == cntTaskLeft-1) {
531         lengthLeft = left->resultSize*lengthLeft;
532     }
533     ioService.post(bind(&Join::histogramTask, this, &ioService, cntTaskLeft, i, 0, startLeft, lengthLeft)); // for left
534 }
535 for (int i=0; i<cntTaskRight; i++) {
536     int startRight = i*lengthRight;
537     if (i == cntTaskRight-1) {
538         lengthLeft = left->resultSize*lengthLeft;
539     }
540     ioService.post(bind(&Join::histogramTask, this, &ioService, cntTaskRight, i, 1, startRight, lengthRight)); // for r
541 }
542
543 void Join::histogramTask(boost::asio::io_service* ioService, int cntTask, int taskIndex, int leftOrRight, unsigned start, u
544     unsigned length) {
545         __sync_synchronize();
546         uint64_t* keyColumn;
547         vector<uint64_t*> inputData = left->getResults();
548         if (!leftOrRight) {
549             inputData = left->getResults();
550             keyColumn = inputData[leftColId];
551         } else {
552             inputData = right->getResults();
553             keyColumn = inputData[rightColId];
554         }
555         for (unsigned i=start; limit=start+length; i<limit; i++) {
556             histograms[leftOrRight][taskIndex][RADIX_HASH(keyColumn[i], cntPartition)]++; // cntPartition으로 나눌 수 있도록 하
557         }
558     }
559 }
```

Basic Linux Command and Tools

#GCC

GCC

- GCC : GNU Compiler Collection
 - An integrated distribution of compilers for several major programming languages, including C, C++, Objective-C/C++, Java, Fortran, and Ada.
 - Compiler generates a binary executable file from a source code written in high level language.
- gcc : GNU C Compiler
 - Compiles *.c / *.cpp files as C and C++ respectively.
- g++ : GNU C++ Compiler
 - *.c / *.cpp files but treats both as C++ files.

Ref) g++ uses standard C++ libraries automatically to link object files, but gcc doesn't. This is why gcc returns ld error during linkage step when compiling with *.cpp files.

Install gcc

```
$ sudo apt update  
$ sudo apt upgrade # If the network is slow, do it next time  
$ sudo apt install gcc
```

```
sp1ab@DESKTOP-HPBRE5B:~$ gcc --version  
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0  
Copyright (C) 2023 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```


Install g++

```
$ sudo apt update  
$ sudo apt upgrade # If the network is slow, do it next time  
$ sudo apt install g++
```

```
splab@DESKTOP-HPBRE5B:~$ g++ --version  
g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0  
Copyright (C) 2023 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

gcc/g++ options

- gcc/g++ [options] <infile> ...
 - -c : compile only, create an object file.
 - -g : compile with debugging information that GDB can use.
 - -o <output file> : place the output file.
 - -I<dir> : add the directory to the list for searching header files.
 - -L<dir> : add the directory to the list for linking library.
 - -D<symbol>[=def] : define a macro to be used in compile.
 - ...

Practice 1

- Compile below hello.c source file using gcc.

```
$ vim hello.c
```

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello\n");  
    return 0;  
}
```

```
splab@DESKTOP-HPBRE5B:~/os$ ls  
splab@DESKTOP-HPBRE5B:~/os$ vim hello.c  
splab@DESKTOP-HPBRE5B:~/os$ ls  
hello.c  
splab@DESKTOP-HPBRE5B:~/os$ gcc -o hello hello.c  
splab@DESKTOP-HPBRE5B:~/os$ ls  
hello hello.c  
splab@DESKTOP-HPBRE5B:~/os$ ./hello  
Hello  
splab@DESKTOP-HPBRE5B:~/os$
```

1-1. Compile + Link: \$ `gcc -o hello hello.c`

1-2. Compile and Link separately:

- \$ `gcc -c hello.o hello.c`
- \$ `gcc -o hello hello.o`

2. Execute: \$ `./hello`

Practice 2

- Compile below two source files using g++.

```
$ vim main.cpp
```

```
void print_hello();
```

```
int main() {  
    print_hello();  
    return 0;  
}
```

```
$ vim print.cpp
```

```
#include <iostream>
```

```
void print_hello() {  
    std::cout << "Hello World" << std::endl;  
}
```

1-1. Compile + Link: \$ `g++ -o hello_world main.cpp print.cpp`

1-2. Compile and Link separately:

- \$ `gcc -c print.o print.cpp`
- \$ `gcc -c main.o main.cpp`
- \$ `gcc -o hello_world main.o print.o`

2. Execute: \$ `./hello_world`

Practice 2

- Compile below two source files using g++.

```
splab@DESKTOP-HPBRE5B:~/os$ ls
hello  hello.c
splab@DESKTOP-HPBRE5B:~/os$ vim main.cpp
splab@DESKTOP-HPBRE5B:~/os$ vim print.cpp
splab@DESKTOP-HPBRE5B:~/os$ ls
hello  hello.c  main.cpp  print.cpp
splab@DESKTOP-HPBRE5B:~/os$ g++ -o hello_world main.cpp print.cpp
splab@DESKTOP-HPBRE5B:~/os$ ./hello_world
Hello World
splab@DESKTOP-HPBRE5B:~/os$ █
```

Basic Linux Command and Tools

#Makefile

Makefile

- Make
 - Build automation tool widely used in Unix-like operating systems.
 - Automatically builds executable programs and libraries from source code by reading Makefile.
- Makefile
 - A file containing a set of directives used with the make build automation tool.
 - Simple command “\$ `make`” can produce executable programs on the basis of Makefile.
- Advantages of Makefile
 - Replace repetitive compile tasks with a simple command.
 - Easy to catch the dependencies of source files in project.

Makefile Manual

- Components of Makefile

```
target: prerequisites
    command1
    command2
    ...
```

- target : file we want to make. (object or executable files)
- prerequisites : necessary file list to make a target file.
- command(s) : sequential steps to make a target file.
- Attention) tab must be in the front of command(s) (Not space)
 - If you configure "set expandtab" in vimrc file, tab would be substituted to spacebar.
 - In this case, press [Ctrl] + [v], then press [tab].

Makefile advanced usage 1

- Macro

- Repeated text can be defined as user-defined Macro.

```
$ vim Makefile
```

```
TARGET = hello_world  
$(TARGET): main.o print.o  
    g++ -o $(TARGET) main.o print.o  
...
```

- First, define the name of macro TARGET.
- In case of referring macro, put a macro in brackets and type '\$' in the front.
- Definition must be ahead of reference.
- Don't use tab in the definition, also "", "", =, : is not allowed.
- There are internally predefined macros (such as CC, CPPFLAG, LD, ...)

Makefile advanced usage 2

- Macro substitution

```
$ vim Makefile
```

```
...  
SRCS = abc.cpp def.cpp ghi.cpp  
OBJS = $(SRCS:.cpp=.o)  
...
```

- More than two text can be defined as a same macro. (using blank)
- Other macro can be defined using the macro which is already defined.
 - Like this : OBJS = abc.o def.o ghi.o

Project Structure

- C/C++ projects in general consist of familiar directories
 - such as bin, include, lib, src.
- Project/
 - bin : consists of output files of compiler
 - include : consists of header files
 - lib : consists of user library files needed for linker
 - src : consists of source files

```
$ sudo apt install tree  
$ tree
```

```
sp1ab@DESKTOP-HPBRE5B:~/os/project$ tree  
.  
├── Makefile  
├── README  
├── bin  
├── include  
├── lib  
└── src  
  
5 directories, 2 files
```

Practice

- Project structure and Makefile example
 - Make a [project] directory.
 - Make a [bin], [include], [lib], [src] directories in [project] directory.
 - Put main.cpp and print.cpp files from previous practice to the [src] directory.

```
sp1ab@DESKTOP-HPBRE5B:~/os/project$ ls
Makefile  README  bin  include  lib  src
sp1ab@DESKTOP-HPBRE5B:~/os/project$ ls src/
main.cpp  print.cpp
sp1ab@DESKTOP-HPBRE5B:~/os/project$
```

- Write the Makefile. (next slide)

Practice

```
splab@DESKTOP-HPBRE5B:~/os/project$ cat Makefile
# Compiler and Compile options.
CC = g++
CXXFLAGS = -g -Wall -std=c++11

# Macros specifying path for compile.
SRCS := $(wildcard src/*.cpp)
OBJS := $(SRCS:.cpp=.o)
BIN = ./bin/
INC = ./include/
LIB = ./lib/ -lpthread

# Pre-Processor
CPPFLAGS += -I$(INC)

#Compile command.
TARGET = print_hello
$(TARGET): $(OBJS)
    $(CC) $(CXXFLAGS) $(CPPFLAGS) -o $(BIN)$@ $(OBJS) -L$(LIB)

# Delete binary & object files.
clean:
    rm $(BIN)$@ $(OBJS)
splab@DESKTOP-HPBRE5B:~/os/project$
```

Practice

```
splab@DESKTOP-HPBRE5B:~/os/project$ ls
Makefile  README  bin  include  lib  src
splab@DESKTOP-HPBRE5B:~/os/project$ ls src/
main.cpp  print.cpp
splab@DESKTOP-HPBRE5B:~/os/project$ make
g++ -g -Wall -std=c++11 -I./include/ -c -o src/main.o src/main.cpp
g++ -g -Wall -std=c++11 -I./include/ -c -o src/print.o src/print.cpp
g++ -g -Wall -std=c++11 -I./include/ -o ./bin/print_hello src/main.o src/print.o -L./lib/ -lpthread
splab@DESKTOP-HPBRE5B:~/os/project$ ./bin/print_hello
Hello World
splab@DESKTOP-HPBRE5B:~/os/project$ make clean
rm ./bin/print_hello src/main.o src/print.o
splab@DESKTOP-HPBRE5B:~/os/project$ █
```